Colombian Collegiate Programming League

CCPL 2026

Round 3 -- April 18

Problems

This set contains 10 problems; pages 1 to 20.

(Borrowed from several sources online.)

A - Equidivisions 1

B - Generalized Matrioshkas 3

C - Babylonian Roulette 5

D - Continuous Fractions 7

E - Polygon Encoder 9

F - Uncle Jack 11

G - Babel Towers 12

H - Little Quilt 14

I - Lazy Professor 16

J - Flight Control 18

Official site `http://programmingleague.org`

Follow us on 𝕏 `@CCPL2003`

A - Equidivisions

Source file name: `equidivisions.c`, `equidivisions.cpp`, `equidivisions.java`, *or* `equidivisions.py`

An equidivision of an $n \times n$ square array of cells is a partition of the n^2 cells in the array in exactly n sets, each one with n contiguous cells. Two cells are contiguous when they have a common side.

A good equidivision is composed of contiguous regions. The figures show a good and a wrong equidivision for a 5×5 square:

1	1	1	5	5
2	1	5	5	4
2	1	5	4	4
2	2	4	4	3
2	3	3	3	3

1	1	1	4	5
2	1	5	4	5
2	1	5	5	4
2	2	4	4	3
2	3	3	3	3

Note that in the second example the cells labeled with 4 describe three non-contiguous regions and cells labeled with 5 describe two non-contiguous regions. You must write a program that evaluates if an equidivision of the cells in a square array is good or not.

Input

It is understood that a cell in an $n \times n$ square array is denoted by a pair (i, j), with $1 \leq i, j \leq n$. The input file contains several test cases. Each test case begins with a line indicating n, $0 < n < 100$, the side of the square array to be partitioned. Next, there are $n - 1$ lines, each one corresponding to one partition of the cells of the square, with some non-negative integer numbers. Consecutive integers in a line are separated with a single blank character. A line of the form

$$a_1\ a_2\ a_3\ a_4 \ldots$$

means that cells denoted with the pairs (a_1, a_2), (a_3, a_4), $\ldots$ belong to one of the areas in the partition. The last area in the partition is defined by those cells not mentioned in the $n - 1$ given lines. If a case begins with $n = 0$ it means that there are no more cases to analyze.

The input must be read from standard input.

Output

For each test case `good` must be printed if the equidivision is good, in other case, `wrong` must be printed. The answers for the different cases must preserve the order of the input.

The output must be written to standard output.

Sample Input	Sample Output
2 1 2 2 1 5 1 1 1 2 1 3 3 2 2 2 2 1 4 2 4 1 5 1 3 1 4 5 5 2 5 3 5 5 5 4 2 5 3 4 3 5 4 3 4 4 5 1 1 1 2 1 3 3 2 2 2 2 1 3 1 4 1 5 1 4 2 4 5 5 2 5 3 5 5 5 4 2 4 1 4 3 5 4 3 4 4 0	wrong good wrong

B - Generalized Matrioshkas

Source file name: `matriosh.c`, `matriosh.cpp`, `matriosh.java`, *or* `matriosh.py`

Vladimir worked for years making *matrioshkas*, those nesting dolls that certainly represent truly Russian craft. A matrioshka is a doll that may be opened in two halves, so that one finds another doll inside. Then this doll may be opened to find another one inside it. This can be repeated several times, till a final doll -that cannot be opened- is reached.

Recently, Vladimir realized that the idea of nesting dolls might be generalized to nesting toys. Indeed, he has designed toys that contain toys but in a more general sense. One of these toys may be opened in two halves and it may have more than one toy inside it. That is the new feature that Vladimir wants to introduce in his new line of toys.

Vladimir has developed a notation to describe how nesting toys should be constructed. A toy is represented with a positive integer, according to its size. More precisely: if when opening the toy represented by m we find the toys represented by $n_1, n_2, \ldots, n_r$, it must be true that $n_1 + n_2 + \cdots + n_r < m$. And if this is the case, we say that toy m *contains directly* the toys $n_1, n_2, \ldots, n_r$. It should be clear that toys that may be contained in any of the toys $n_1, n_2, \ldots, n_r$ are not considered as directly contained in the toy m.

A *generalized matrioshka* is denoted with a non-empty sequence of non zero integers of the form:

$$a_1\ a_2 \ldots a_N$$

such that toy k is represented in the sequence with two integers $-k$ and k, with the negative one occurring in the sequence first that the positive one.

For example, the sequence

$$-9\ 7\ -2\ 2\ -3\ -2\ -1\ 1\ 2\ 3\ 7\ 9$$

represents a generalized matrioshka conformed by six toys, namely, 1, 2 (twice), 3, 7 and 9. Note that toy 7 contains directly toys 2 and 3. Note that the first copy of toy 2 occurs left from the second one and that the second copy contains directly a toy 1. It would be wrong to understand that the first -2 and the last 2 should be paired.

On the other hand, the following sequences do not describe generalized matrioshkas:

- $$-9\ -7\ -2\ 2\ -3\ -1\ -2\ 2\ 1\ 3\ 7\ 9$$

  because toy 2 is bigger than toy 1 and cannot be allocated inside it.

- $$-9\ -7\ -2\ 2\ -3\ -2\ -1\ 1\ 2\ 3\ 7\ -2\ 2\ 9$$

  because 7 and 2 may not be allocated together inside 9.

- $$-9\ -7\ -2\ 2\ -3\ -1\ -2\ 3\ 2\ 1\ 7\ 9$$

  because there is a nesting problem within toy 3.

Your problem is to write a program to help Vladimir telling good designs from bad ones.

Input

The input file contains several test cases, each one of them in a separate line. Each test case is a sequence of non zero integers, each one with an absolute value less than 10^7.

The input must be read from standard input.

Output

Output texts for each input case are presented in the same order that input is read.

For each test case the answer must be a line of the form

`:-) Matrioshka!` if the design describes a generalized matrioshka. In other case, the answer should be of the form

`:-( Try again.`

The output must be written to standard output.

Sample Input	Sample Output
-9 -7 -2 2 -3 -2 -1 1 2 3 7 9	:-) Matrioshka!
-9 -7 -2 2 -3 -1 -2 2 1 3 7 9	:-(Try again.
-9 -7 -2 2 -3 -1 -2 3 2 1 7 9	:-(Try again.
-100 -50 -6 6 50 100	:-) Matrioshka!
-100 -50 -6 6 45 100	:-(Try again.
-10 -5 -2 2 5 -4 -3 3 4 10	:-) Matrioshka!
-9 -5 -2 2 5 -4 -3 3 4 9	:-(Try again.

C - Babylonian Roulette

Source file name: `broul.c`, `broul.cpp`, `broul.java`, *or* `broul.py`

... Los babilonios se entregaron al juego. El que no adquiría suertes era considerado un pusilánime, un apocado. Con el tiempo, ese desdén justificado se duplicó. Era despreciado el que no jugaba, pero también eran despreciados los perdedores que abonaban la multa ...

Jorge Luis Borges, La lotera de Babilonia, Ficciones

People of Babylon were devoted to chance games and one of the most popular was a special kind of roulette. Recently, some old Babylonian tablets were found. They described details of the roulette game.

In modern terms, the rules of the game were as follows:

- Roulette's compartments had only six labels: $-1, -2, -3, 1, 2, 3$.
- The game was played by turns, during a day. Turns were numerated $0, 1, 2, \ldots$
- Players could win or lose a multiple of the *bet*, a quantity of money that was constant along the day.
- At turn t there was an amount of money P_t, called the *pot*.
- At the start, there was an initial amount of money P_0 in the pot.
- P_0 and the bet were positive numbers arbitrarily defined by the King.
- In a turn, a player turned the roulette. A player could not play more than once in a day. Depending on the compartment where the ball came to rest, the player lose (or won, if the value was negative) an amount $w_t = L \cdot bet$ of money, where L corresponded to the compartment's label.
- The won money was taken from the pot (or put in it if the player lose), i.e. the value of the pot in a given turn was determined by $P_{t+1} = P_t + w_t$.
- If as a result of the last rule P_{t+1} was a negative number the winner won only the maximum multiple of the bet that he could win without making a negative pot.
- If at some turn the pot was less than the bet, the game was end for that day. If that was not the case the game continued till sunset.

Beside the tablets that explained the rules some other tablets were found. These had lines with three numbers. Archeologists conjecture that each of these lines were part of a kind of accountability system for the game, where numbers represented, for a given day, the value of the pot at the beginning, the bet and the value of the pot at the end.

For example, a line with the numbers

10000 1500 11500

could mean that there was only one turn where the player lose with label 1. Another possibility is that there were three turns with results 2, 1 and -2.

On the other hand, there were found other tablets with triplets of numbers that seem like the above described that, however, cannot represent results of a game day. There is no hypothesis of what they are.

Archeologists want to validate their hypothesis analyzing batches of tablets with triplets. They want to estimate the number of people that played in a day. To begin, they want to establish, for each triplet of numbers in a tablet that could represent a result of a game day, the minimal number of players that played that day. In the above

example the answer to this question is 1. Tablets that cannot represent results should be identified. You are hired to help with this task.

Input

The input

le contains several test cases, each one of them in a separate line. Each test case is a triplet of non negative integers, indicating the initial pot, the bet and the final pot for a day. Each of the input numbers is less than 10^8. The initial pot and the bet are greater than 0.

A line with a triplet of 0's denotes the end of the input.

The input must be read from standard input.

Output

Output texts for each input case are presented in the same order that input is read. For each test case the answer must be a printed line.

If the test case cannot represent the result of a game day, the output line has the words `No accounting tablet`. In other case, the printed answer is one positive integer number telling the minimal number of players that could turn the roulette for the day corresponding to the annotations.

The output must be written to standard output.

Sample Input	**Sample Output**
10000 1000 22000 24 13 2 5100 700 200 54 16 158 360 6 72 25 10 5 0 0 0	4 No accounting tablet 3 No accounting tablet 16 1

D - Continuous Fractions

Source file name: `cfrac.c`, `cfrac.cpp`, `cfrac.java`, *or* `cfrac.py`

A simple continuous fraction has the form:

$$a_1 + \cfrac{1}{a_2 + \cfrac{1}{a_3 + \cfrac{1}{\ddots + \cfrac{1}{a_n}}}}$$

where the a_i's are integer numbers.

The previous continuous fraction could be noted as $[a_1, a_2, \ldots, a_n]$. It is not difficult to show that any rational number $\frac{p}{q}$, with integers $p > q > 0$, can be represented in a unique way by a simple continuous fraction with n terms, such that $\frac{p}{q} = [a_1, a_2, \ldots, a_n, 1]$, where n and the a_i's are positive natural numbers.

Your task is to

nd and print the simple continuous fraction that corresponds to a given rational number.

Input

Input will consist of a series of cases, each one in a line. A line describing a case contains p and q, two integer numbers separated by a space, with $10^{20} > p > q > 0$.

The end of the input is indicated by a line containing `0 0`.

The input must be read from standard input.

Output

Cases must be analyzed in the order that are read from the input. Output for each case will consist of several lines. The first line indicates the case number, starting at 1, using the format:

`Case` *i* `:`

replacing *i* by the corresponding case number.

The second line displays the input data in the form p / q.

The remaining lines must contain the continuous fraction corresponding to the rational number, $\frac{p}{q}$, specified in the given input line. The continuous fraction must be printed accordingly to the following rules:

- Horizontal bars are formed by sequences of dashes `-`.
- The width of each horizontal bar is exactly equal to the width of the denominator under it.
- Blank characters should be printed using periods `.`.
- The number on a fraction numerator must be printed center justified. That is, the number of spaces at either side must be same, if possible; in other case, one more space must be added at the right side.

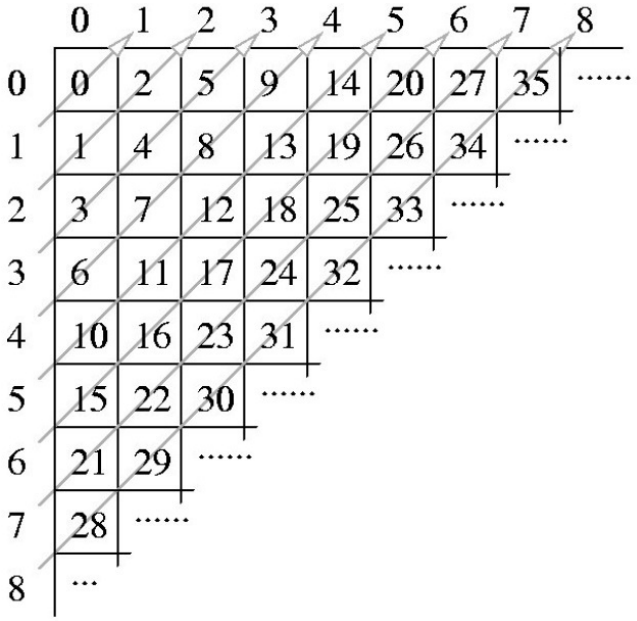
The output must be written to standard output.

Sample Input	Sample Output
75 34 65 60 0 0	Case 1: 75 / 34 ....1..... 2.+.---------- .....1.... ....4.+.--------- .....1.. .....1.+.----- .....1 .....5.+.- .....1 Case 2: 65 / 60 .....1... 1.+.------ .....1 ....11.+.- 1

E - Polygon Encoder

Source file name: `polygon.c`, `polygon.cpp`, `polygon.java`, *or* `polygon.py`

Imagine an ifinite table with rows and columns numbered using the natural numbers. The following figure shows a procedure to traverse such a table assigning a consecutive natural number to each table cell:



This enumeration of cells can be used to represent complex data types using natural numbers:

- A pair of natural numbers (i, j) is represented by the number corresponding to the cell in row i and column j. For instance, the pair $(3, 2)$ is represented by the natural number 17; this fact is noted by $P_2(3, 2) = 17$.
- The pair representation can be used to represent n-tuples. A triplet (a, b, c) is represented by $P_2(a, P_2(b, c))$. A 4-tuple (a, b, c, d) is represented by $P_2(a, P_2(b, P_2(c, d)))$. This procedure can be generalized for an arbitrary n:

  $$P_n(a_1, \ldots, a_n) = P_2(a_1, P_{n-1}(a_2, \ldots, a_n))$$

  where P_n denotes the n-tuple representation function, $n \geq 2$. For example $P_3(2, 0, 1) = 12$.
- A list of arbitrary length $\langle a_1, \ldots, a_n \rangle$ is represented by

  $$L(\langle a_1, \ldots, a_n \rangle) = P_2(n; Pn(a_1, \ldots, a_n)).$$

  For example, $\mathrm{L}(\langle 0; 1 \rangle) = 12$.

The Association of Convex Makers (ACM) uses this clever enumeration scheme in a polygon representation system. The system can represent a polygon, defined by integer coordinates, using a natural number as follows: given a polygon defined by a vertex sequence $\langle (x_1, y_1), \ldots, (x_n, y_n) \rangle$ assign the natural number:

$$L(\langle P_2(x_1, y_1), \ldots, P_2(x_n, y_n) \rangle).$$

ACM needs a program that, given a natural numbers that represents a polygon, calculates the area of the polygon. It is guaranteed that the given polygon is a simple one, i.e. its sides do not intersect.

As an example of the problem, the triangle with vertices at $(1, 1)$, $(2, 0)$ and $(0, 0)$ is codified with the number 2141. The area of this triangle is 1.

Input

The input consists of several test cases. Each test case is given in a single line of the input by a natural number representing a polygon. The end of the test cases is indicated with *.

The input must be read from standard input.

Output

One line per test case, preserving the input order. Each output line contains a decimal number telling the area of the corresponding encoded polygon. Areas must be printed with 1 decimal place, truncating less significative decimal places.

The output must be written to standard output.

Sample Input	**Sample Output**
2141 206 157895330 *	1.0 0.5 1.0

F - Uncle Jack

Source file name: `uj.c`, `uj.cpp`, `uj.java`, *or* `uj.py`

Dear Uncle Jack is willing to give away some of his collectable CDs to his nephews. Among the titles you can find very rare albums of Hard Rock, Classical Music, Reggae and much more; each title is considered to be unique. Last week he was listening to one of his favorite songs, *Nobody's fool*, and realized that it would be prudent to be aware of the many ways he can give away the CDs among some of his nephews.

So far he has not made up his mind about the total amount of CDs and the number of nephews. Indeed, a given nephew may receive no CDs at all.

Please help dear Uncle Jack, given the total number of CDs and the number of nephews, to calculate the number of different ways to distribute the CDs among the nephews.

Input

The input consists of several test cases. Each test case is given in a single line of the input by, space separated, integers N ($1 \leq N \leq 10$) and D ($0 \leq D \leq 25$), corresponding to the number of nephews and the number of CDs respectively. The end of the test cases is indicated with $N = D = 0$.

The input must be read from standard input.

Output

The output consists of several lines, one per test case, following the order given by the input. Each line has the number of all possible ways to distribute D CDs among N nephews.

The output must be written to standard output.

Sample Input	Sample Output
1 20 3 10 0 0	1 59049

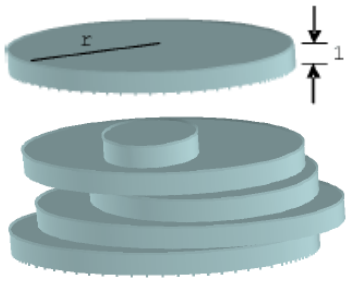
G - Babel Towers

Source file name: `babel.c`, `babel.cpp`, `babel.java`, *or* `babel.py`

Babel Inc. is a company that designs structures for skyscrapers. They have developed a simple technique to estimate the feasibility of a design, based on the fact that their designs are equivalent to stack blocks of circular section and constant height. The height of a block is used as the unit of lineal measure.

Blocks are all made of the same non deformable uniform material, so that the weight of a block is proportional to its volume and its center of mass is the geometrical center of the block. A typical block is of the form which weight is determined by the radius of the circular section (because every one has height 1).

For $n \geq 1$, an n-tower is a stack of n blocks. Blocks in an n-tower are numbered $0, 1, \ldots, n-1$, from the bottom to the top. Given an n-tower, its k-subtower is defined as the k-tower with the first k blocks of the n-tower, $0 \leq k < n$. An example of a 5-tower could look like the following figure:



A tower is *feasible* when it can be built putting its blocks one by one from bottom to top. At each building step the corresponding subtower should be *stable*, i.e., every block of it must remain in the position that it is designed to stay. When placing a block results in a stack of blocks which part of it has a center of mass that lies out of the base on that it is supported we say that the resulting tower *collapses* and is *unfeasible*.

Babel Inc. does not consider stable a situation where the center of mass of the system of blocks that are supposed to rest on a block lies on the border of the supporting surface. For instance, two blocks of the same dimensions with one of them placed centered on the circumference of the other constitute a 2-tower that collapses; but if the second one is placed within the circle of the first one the system is feasible.

Your problem is to help Babel Inc. in the evaluation of tower designs. Given a design for a n-tower you must judge if it is feasible and, if it is not, you must tell at which floor it would collapse. In this last case you give k as answer, with $0 < k < n$, if the $(k-1)$-subtower is feasible but the k-subtower collapses.

Input

The input file contains several test cases.

Each test case begins with a line indicating N, $0 < N < 1000$, the number of blocks that the designed tower should have.

Then, there is a line describing each one of the blocks. The block k, $0 \leq k < N$, is described by a line containing 3 integers separated by blanks, of the form

$$x_k \; y_k \; r_k$$

which indicates that the block k with radius r_k must be placed supported on the block $k-1$ (exception: block 0 is placed on the ground) with its center of mass on the point $\langle x_k, y_k, k + 0.5 \rangle$ with respect to a grid with origin in $\langle 0, 0, 0 \rangle$, for $x_k, y_k \leq \left|10^5\right|$ and $1 \leq r_k \leq 10^5$.

A case begin with $N = 0$ means that there are no more cases to analyze.

The input must be read from standard input.

Output

Output texts for each input case are presented in the same order that input is read.

For each test case the answer must be of the form `Feasible` if the design is feasible. In other case, the answer should be of the form `Unfeasible` k indicating that the $(k-1)$-subtower is feasible but the k-subtower is unfeasible.

The output must be written to standard output.

Sample Input	**Sample Output**
3 0 0 10 2 0 12 -4 1 1 4 0 0 12 0 0 10 0 9 10 0 17 5 4 0 0 4 0 1 4 1 0 4 -1 -1 4 2 10 10 5 0 0 3 0	Feasible Unfeasible 3 Feasible Unfeasible 1

H - Little Quilt

Source file name: `quilt.c`, `quilt.cpp`, `quilt.java`, *or* `quilt.py`

Little Quilt is a small language introduced by Ravi Sethi in his book 'Programming Languages'. Here, a restricted version of Little Quilt is presented.

The language is defined by the following BNF grammar:

$$< \texttt{QUILT} >::= \texttt{A}|\texttt{B}|\texttt{turn}(< \texttt{QUILT} >)|\texttt{sew}(< \texttt{QUILT} >, < \texttt{QUILT} >)$$

`A` and `B` represent the two primitive quilts. Each primitive quilt corresponds to a matricial arrangement of 2×2 characters. `turn()` and `sew()` are operations over quilts.

The instruction `turn(x)` turns the quilt `x` 90 degrees clockwise. The following table illustrates the primitive quilts as well as examples of the effect of the `turn()` operation:

A	// /+
turn(A)	\\ +\
turn(turn(A))	+/ //
turn(turn(turn(A)))	\+ \\
B	-- --
turn(B)	|| ||

Accordingly, the instruction `sew(x,y)` sews quilt `x` to the left of quilt `y`. Both `x` and `y` must have the same height, otherwise an error will be generated. The following figure represents the result of `sew(A,turn(B))`:

```
//||
/+||
```

while the `sew(turn(sew(B,turn(B))),A)` generates an error message.

Your job is to build an interpreter of the Little Quilt language.

Input

The input file will be a text file containing different Little Quilt expressions, each one ended by a semicolon character (`;`). Space and new line characters must be ignored; this means that an expression may span several lines.

The input must be read from standard input.

Output

The output file contains the quilts produced as a result of interpreting the input expressions.

Each quilt must be preceded by a line, left aligned, with the format `Quilt i:` where *i* is the quilt number, starting at 1. If the expression interpretation generates and error, the word `error` must be printed.

The output must be written to standard output.

Sample Input	**Sample Output**

`sew(turn(sew(B,turn(B))),`
`turn(sew(turn(B),B)))  ;`
 	
`sew(turn(sew(B,turn(B))),A);`
`sew(turn(sew(A,turn(A))),`
`turn(turn(`
 	
`turn(sew(A,turn(A))))))`
 	
`;`
	`Quilt 1:`
	`||--`
	`||--`
	`--||`
	`--||`
	`Quilt 2:`
	`error`
	`Quilt 3:`
	`\\//`
	`+\/+`
	`+/\+`
	`//\\`

I - Lazy Professor

Source file name: `lazyprof.c`, `lazyprof.cpp`, `lazyprof.java`, *or* `lazyprof.py`

Professor Yzal does not like to grade the exams of his students, so that he assigns this task to his assistants. Final students' grades are defined as weighted sums of the grades obtained in the exams. But Professor Yzal does not define *a priori* the exam's relative weights. Instead of that, only when the assistants end their job (and all the students' grades in each exam are known) Professor Yzal decides the exam's weights. He does it trying to be pleasant with the whole class, so that the assigned weights should maximize the average final grade that the class obtains, and expecting that eventual complaints about his grading criteria will be as few as possible. And finally, maybe rewarding harder exams, Yzal defines that the weight of every particular exam should lie within a specific range of values.

This term is about to finish and you were hired to help Yzal with his lazy grading job.

Your task is to write a program that, given the students' grades and the *reasonable range* of weights for each exam, determines the maximum possible average according to the following rules:

- Each exam must have a weight, defined as an integer number in the closed interval $[0, 100]$, describing the assigned percentage.
- The weight of an exam i must be in an integer closed interval $[min_i, max_i]$ that describes the corresponding reasonable range previously defined by Yzal.
- The sum of all exam weights must be 100.
- The final grade of each student is a weighted sum calculated as the sum of his/her own grades previously multiplied by the corresponding exam weight divided by 100.
- The exam weights are so chosen that the average of the students' final grades is maximized.

Input

There are several cases to consider. Each test case is described as follows:

- A line with two integer numbers S and N, separated by a blank, ($1 \le S \le 100$, $1 \le N \le 20$), where S corresponds to the number of students and N corresponds to the number of exams in the course.
- Each one of the following S lines describes the grades of each student, containing N integer numbers $c_{j1}, c_{j2}, \ldots, c_{jN}$, separated by a blank, ($0 \le c_{ji} \le 100$ for each $1 \le i \le N$), where c_{ji} indicates the grade obtained by the student j in the exam i ($1 \le j \le S$, $1 \le i \le N$).
- Each one of the following N lines describes the reasonable range of weights of each exam, containing two integer numbers min_i, max_i, separated by a blank ($0 \le min_i \le max_i \le 100$), where min_i and max_i represent the minimum and maximum weight that can be assigned to the exam i, respectively ($1 \le i \le N$).

You can suppose that

$$\sum_{i=1}^{N} min_i \le 100, \quad \text{and} \quad \sum_{i=1}^{N} max_i \ge 100.$$

The last test case is followed by a line containing two zeros.

The input must be read from standard input.

Output

For each given case, output a single line with a number indicating the maximum possible average of the students' final grades if the weights are defined according to the rules. The answer should be formatted and approximated to two decimal places. The floating point delimiter must be '.' (i.e., the dot). The rounding applies towards the *nearest neighbor* unless both neighbors are equidistant, in which case the result is rounded up (e.g., 78.312 is rounded to 78.31; 78.566 is rounded to 78.57; 78.345 is rounded to 78.35, etc.).

The output must be written to standard output.

Sample Input	Sample Output
1 1 0 0 100 2 2 50 90 70 50 0 100 0 100 2 2 50 90 70 50 30 70 30 70 2 2 50 90 70 50 50 50 50 50 2 2 73 52 92 81 20 50 60 80 0 0	0.00 70.00 67.00 65.00 72.90

J - Flight Control

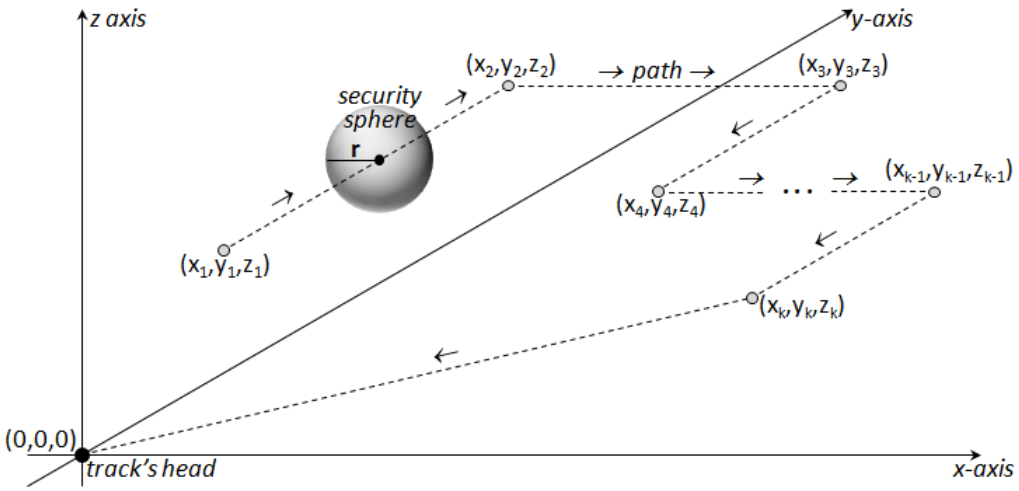
Source file name: `flight.c`, `flight.cpp`, `flight.java`, *or* `flight.py`

Air traffic controllers are the people who expedite and maintain a safe and orderly flow of air traffic in the global air traffic control system. The position of the air traffic controller is one that requires highly specialized skills. Because controllers have an incredibly large responsibility while on duty, this profession is, according to Wikipedia, ''regarded around the world as one of the most difficult jobs today, and can be notoriously stressful depending on many variables (equipment, configurations, weather, traffic volume, human factors, etc.)''.

An air traffic controller has access to the following information for each aircraft on the screen:

- *size*: a positive integer number r indicating the radius (measured in meters) of a *security sphere* whose center always is the current position of the aircraft;
- *speed*: a positive integer number s indicating the constant speed (measured in meters per second) of the aircraft along its route;
- *route*: a sequence of points $(x_1, y_1, z_1), (x_2, y_2, z_2), \ldots, (x_k, y_k, z_k)$ with integer coordinates (measured in meters) in the three-dimensional Cartesian plane, indicating the path followed by the aircraft before it returns to the head of the *track* which is located at the point $(0, 0, 0)$.

Each aircraft begins its journey at the position (x_1, y_1, z_1) and then, it directly flies in a straight line at constant speed from (x_1, y_1, z_1) to (x_2, y_2, z_2), ..., from $(x_{k-1}, y_{k-1}, z_{k-1})$ to (x_k, y_k, z_k), and finally from (x_k, y_k, z_k) to $(0, 0, 0)$, where each position is relative to the head of the track. After the aircraft arrives at the head of the track, it disappears from the controller's screen.



On a day-to-day basis, air traffic controllers deal with conflict detection and resolution. Therefore, for an air traffic controller is very important to have an alarm system to indicate the specific points where it must take corrective actions to prevent accidents.

It is noteworthy that, geometrically speaking, a *conflict warning* occurs when the security spheres of two aircrafts touch. Formally, a *conflict warning* begins when two aircrafts approach at a distance less than or equal to the sum of the radius of their security spheres, is maintained while this condition is satisfied, and ends when their security spheres stop touching (i.e., when the distance between both is greater than the sum of the radius of their security spheres). Distances are measured with an acceptable error threshold $\varepsilon > 0$.

Despite years of effort and the billions of dollars that have been spent on computer software designed to assist air traffic control, success has been largely limited to improving the tools at the disposal of the controllers. However, today you have the chance to improve the impact of computer software in the air traffic control world.

You have been hired by the *International Center of Planning Control* (ICPC) to determine the quality of the traffic routes defined by the *Aircraft Controller Management* (ACM), through the measurement of the number of dangerous situations that should fix the air traffic controller. Your task is to write a program that, given the information of two aircrafts, determines the number of different conflict warnings that would arise if both aircrafts follow the scheduled route starting at the same time and finishing at the track's head.

Input

The first line of the input contains the number of test cases. Each test case specifies the information of the two studied aircrafts, where each aircraft is described as follows:

- The first line contains three integer numbers r, s and k separated by blanks ($1 \leq r \leq 100$, $1 \leq s \leq 1000$, $1 \leq k \leq 100$), where r is the radius of the security sphere (in meters), s is the speed (in meters per second), and k is the number of points that define the route of the aircraft.
- Each one of the next k lines contains three integer numbers x_i, y_i, and z_i separated by blanks ($-10^4 \leq x_i \leq 10^4$, $-10^4 \leq y_i \leq 10^4$, $0 \leq z_i \leq 10^4$), describing the coordinates (in meters) of the i-th point on the route of the aircraft ($1 \leq i \leq k$).

For each route, you may assume that either $x_i \neq x_{i+1}$, $y_i \neq y_{i+1}$ or $z_i \neq z_{i+1}$ for all $1 \leq i < k$, and that either $x_k \neq 0$, $y_k \neq 0$ or $z_k \neq 0$. The acceptable error threshold is $\varepsilon = 10^{-10}$ meters.

The input must be read from standard input.

Output

For each test, print one line informing the number of different conflict warnings.

The output must be written to standard output.

Sample Input	Sample Output
7 20 300 2 10000 1000 5000 1000 100 500 20 100 3 10000 1000 2000 1000 100 500 100 0 0 20 300 2 0 10000 5000 0 -10000 5000 20 300 3 10000 0 5010 -10000 0 5010 -8000 1000 3010 20 300 2 0 10000 5000 0 -10000 5000 20 300 2 10000 0 5010 -10000 0 5010 25 200 2 3000 6000 3000 4000 5000 3000 25 200 2 3000 6000 3005 4000 5000 3005 20 300 2 5000 4000 3000 4000 5000 3000 20 300 2 3000 6000 3005 4000 5000 3005 10 100 1 -1000 0 0 10 100 2 1000 21 0 -1000 21 0 10 100 1 -1000 0 0 10 100 2 1000 20 0 -1000 20 0	0 1 2 1 1 0 1